Tourist Attraction Recommender System using content based filtering

A Project Report

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Deeksha Singh

(Admission No: 25SCS1003004511)

Batch: 1CSE31

Under the supervision of

Dr. LALITA CHAUDHARY Assistant Professor, SCSE IILM University, Greater Noida

(November 2025)

ABSTRACT

This project presents a Tourist Attraction Recommender System that helps users discover places that match their interests. The system uses Content-Based Filtering, where recommendations are made by comparing a user's preferences with the features of different tourist attractions such as location, category, activities, and popularity. The model analyzes keywords provided by the user and calculates similarity scores to suggest the most suitable destinations. This AI-based approach makes travel planning easier, more personalized, and time-saving for tourists. The system can be extended by adding user ratings, reviews, or integrating collaborative filtering for improved accuracy**.**

Keywords: Content-Based Filtering,Tourist Attraction Recommendation,Machine Learning,Similarity Score,Feature Extraction,Personalized Suggestions,Travel Recommendation System

Contents

- **Abstract**
- **1. Introduction**
  - 1.1 Overview
  - 1.2 Objectives
  - 1.3 Problem Statement
- **2. Literature Review**
- **3. Methodology**
  - 3.1 Architecture Overview
  - 3.2 Hardware Components
  - 3.3 Software Tools
- **4. Implementation**
- **5. Results and Discussion**
- **6. Conclusion and Future Scope**
  - 6.1 Conclusion
  - 6.2 Future Scope
- **References**

1.1 INTRODUCTION

A Tourist Attraction Recommender System helps users find places that match their interests using AI and Machine Learning. In this project, Content-Based Filtering is used to compare user preferences with the features of different tourist spots, such as location and type of activity. Based on similarity, the system suggests the most suitable destinations. This makes travel planning

easier, faster, and more personalized for users.

1.2 Objectives

The project aims to achieve the following objectives:

- Data Collection: Collect data of the tourist attractions like location, type, cost, ratings, etc .

- Use AI to categories historical, adventure, spiritual, leisure, etc. Apply TF-IDF or similarity metrics for

textual data.

- Recommendation Model: Build a content based on the interest of the tourist. Suggest the to attractions for a given tourist profile.

- Evaluation & Visualization: Evaluate recommendations manually. Display results in a simple way in

form of table.

1.3 Problem Statement

To design a lightweight content-based recommendation system that suggests tourist attractions based

on similarity in features (location, category, cost, and ratings). It will be budget friendly.

2. Literature Review

Existing tourist recommendation systems mostly use Collaborative Filtering, which requires large user data. However, Content-Based Filtering works better when user history is limited because it matches attractions with user preferences using features like location, category, cost, and ratings. Research also supports lightweight and low-cost models that give accurate suggestions without heavy processing. These systems are useful for budget-friendly travel planning.

3. Methodology

1. Data Collection: Gather tourist attraction data (location, category, cost, ratings).

2. Feature Processing: Encode features and normalize numerical values.

3. Similarity Calculation: Use cosine similarity to compare user preferences with attraction features.

4. Content-Based Filtering: Generate recommendations based on similarity scores.

5. Ranking: Sort results from most to least similar.

6. Output: Display top budget-friendly tourist attractions.

3.1 Architecture Overview

The system follows a simple Content-Based Recommendation Architecture.

User preferences (location, category, budget) → Feature Matching Module → Similarity Calculator → Recommendation Engine → Suggested tourist attractions.

The flow is lightweight and fast, suitable even for basic devices.

3.2 Hardware Components

Computer/Laptop (for running the model)

Basic Processor (Intel i3 or equivalent)

4–8 GB RAM

Storage: For dataset (few MBs only)

(No special hardware like GPU is required.)

3.3 Software Tools

Python

Jupyter Notebook / VS Code

Scikit-learn (for similarity and ML functions)

Pandas & NumPy (for dataset handling)

Matplotlib (optional for visualization)

IMPLEMENTATION

Here is the clean and simple Python code only, with Author: Deeksha Singh included as a comment:

```
# ----------------------------------------------
# Tourist Attraction Recommender System (Content-Based)
# Author: Deeksha Singh
# ----------------------------------------------

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics.pairwise import cosine_similarity
from scipy.sparse import hstack
from scipy import sparse
import numpy as np

# Sample dataset
data = [
    {"id":1, "name":"Red Fort", "location":"Delhi", "category":"Historical, Monument", "cost":50, "rating":4.4},
    {"id":2, "name":"Marine Drive", "location":"Mumbai", "category":"Beach, Scenic", "cost":0, "rating":4.5},
    {"id":3, "name":"Golden Temple", "location":"Amritsar", "category":"Religious, Monument", "cost":10, "rating":4.8},
    {"id":4, "name":"Lonavala Lake", "location":"Lonavala", "category":"Nature, Scenic", "cost":20, "rating":4.1},
    {"id":5, "name":"Hawa Mahal", "location":"Jaipur", "category":"Palace, Historical", "cost":30, "rating":4.3},
```

```
    {"id":6, "name":"Varkala Beach", "location":"Kerala", "category":"Beach, Relaxation", "cost":0, "rating":4.6},
    {"id":7, "name":"Qutub Minar", "location":"Delhi", "category":"Historical, Monument", "cost":40, "rating":4.2},
    {"id":8, "name":"Rishikesh River", "location":"Rishikesh", "category":"Adventure, Nature", "cost":15, "rating":4.7},
    {"id":9, "name":"Kodaikanal Lake", "location":"Kodaikanal", "category":"Nature, Scenic", "cost":25, "rating":4.4},
    {"id":10,"name":"City Museum", "location":"Pune", "category":"Museum, Educational", "cost":20, "rating":4.0},
]

df = pd.DataFrame(data)

# Combine location + category text
df["text_feat"] = df["location"] + " " + df["category"]

# TF-IDF vectorization
vectorizer = TfidfVectorizer()
text_matrix = vectorizer.fit_transform(df["text_feat"])

# Scale numeric features
scaler = MinMaxScaler()
numeric_matrix = scaler.fit_transform(df[["cost", "rating"]])
numeric_sparse = sparse.csr_matrix(numeric_matrix)

# Final combined feature matrix
feature_matrix = hstack([text_matrix, numeric_sparse])
```

```
# Build user preference vector
def build_user_vector(preferred_location="", preferred_categories=None,
budget=None, min_rating=None):
    if preferred_categories is None:
        preferred_categories = []

    # Prepare text input
    text = preferred_location + " " + ", ".join(preferred_categories)
    text_vec = vectorizer.transform([text])

    # Numeric inputs
    cost_val = np.median(df["cost"]) if budget is None else budget
    rating_val = np.median(df["rating"]) if min_rating is None else min_rating

    numeric_scaled = scaler.transform([[cost_val, rating_val]])
    numeric_sparse_pref = sparse.csr_matrix(numeric_scaled)

    # Combined vector
    return hstack([text_vec, numeric_sparse_pref])

# Recommend function
def recommend(user_vector, top_n=5):
    sims = cosine_similarity(user_vector, feature_matrix).flatten()
    df["score"] = sims
    results = df.sort_values("score", ascending=False).head(top_n)
    return results[["id","name","location","category","cost","rating","score"]]
```

```
# Example user input
user_pref = build_user_vector(
    preferred_location="Delhi",
    preferred_categories=["Historical", "Monument"],
    budget=50,
    min_rating=4.0
)

# Get recommendations
top_recommendations = recommend(user_pref)
print(top_recommendations)
```

5. Results and Discussion

The proposed Tourist Attraction Recommender System successfully provides personalized suggestions using a lightweight Content-Based Filtering approach. When users enter their preferences (location, category, cost, rating), the system calculates similarity and ranks attractions accordingly.

The results show that the model accurately recommends places related to the user's interests, even with a small dataset. Since the model does not depend on large historical data, it performs efficiently and remains fast, budget-friendly, and easy to run on basic hardware.

Conclusion

This project demonstrates that a lightweight content-based recommendation system can effectively suggest tourist attractions based on user preferences. By using features like location, category, cost, and ratings, the system provides relevant and personalized travel options. The model is simple, efficient, and suitable for users who need quick travel recommendations without requiring heavy computational resources.

Future Scope

Hybrid Recommendation System: Combine content-based and collaborative filtering for better accuracy.

Real-Time User Feedback: Use ratings and reviews to improve suggestions.

Expanded Dataset: Add more cities, tourist spots, images, and descriptions.

Mobile App / Web App Integration: Develop an interactive platform for real users.

Map Integration: Show routes, distance, and nearby attractions using APIs.

Budget Optimization: Suggest complete travel plans within a user's budget.

References

1. J. Bobadilla, F. Ortega, A. Hernando, "Recommender Systems Survey," Knowledge-Based Systems, 2013.

2. Ricci, F., Rokach, L., Shapira, B., "Introduction to Recommender Systems," Springer, 2011.